# RIMRULE: Improving Tool-Using Language Agents via MDL-Guided Rule Learning

**Xiang Gao**[1] **Yuguang Yao**[1] **Qi Zhang**[2] **Kaiwen Dong**[1]
**Avinash Baidya**[1] **Ruocheng Guo**[1] **Hilaf Hasson**[1] **Kamalika Das**[1]

[1]Intuit AI Research
[2]Temple University

{xiang_gao, kamalika_das}@intuit.com

arXiv:2601.00086v3 [cs.CL] 8 Jul 2026

## Abstract

Large language models (LLMs) often struggle to use tools reliably in domain-specific settings, where APIs may be idiosyncratic, under-documented, or tailored to private workflows. This highlights the need for effective adaptation to task-specific tools. We propose RIMRULE, a neuro-symbolic approach for LLM adaptation based on dynamic rule injection. Compact, interpretable rules are distilled from failure traces and injected into the prompt during inference to improve task performance. These rules are proposed by the LLM itself and consolidated using a Minimum Description Length (MDL) objective that favors generality and conciseness. Each rule is stored in both natural language and a structured symbolic form, supporting efficient retrieval at inference time. Experiments on tool-use benchmarks show that this approach improves accuracy on both seen and unseen tools without modifying LLM weights. It outperforms prompting-based adaptation methods and complements finetuning. Moreover, rules learned from one LLM can be reused to improve others, including long reasoning LLMs, highlighting the portability of symbolic knowledge across architectures.[1]

## 1 Introduction

Humans adapt by trying, erring, and compressing experience into reusable heuristics (Kolb, 1984; Metcalfe, 2017; Gigerenzer and Gaissmaier, 2011). Such compact representation of regularities achieve cognitive economy—they preserve what matters for performance while discarding incidental detail, making knowledge easier to reuse across tasks and by other people (Rosch, 1978).

Large language models (LLMs) (Brown et al., 2020; Bai et al., 2023; Team and Others, 2023; Grattafiori et al., 2024) also require adaptation, especially when deployed in unfamiliar domains with specialized tools or under-documented APIs. However, the way LLMs adapt differs markedly from how humans do. Today, adaptation typically takes one of three forms: (i) retrieved examples for few-shot prompting (Brown et al., 2020; Liu et al., 2021), (ii) globally tuned prompts (Pryzant et al., 2023; Cui et al., 2025), or (iii) fine-tuned model weights (Hu et al., 2022). Each approach is powerful, but none supports abstraction, reuse, and interpretability in the way humans generalize from experience. Few-shot prompting reuses raw supervision but does not abstract—and LLMs often extract only shallow patterns from examples (Wei et al., 2023); global prompts are static and brittle in interactive settings (Verma et al., 2024); and tuning model weights is costly, requiring retraining whenever the environment changes, and preventing easy sharing of acquired knowledge across models (Yang et al., 2024).

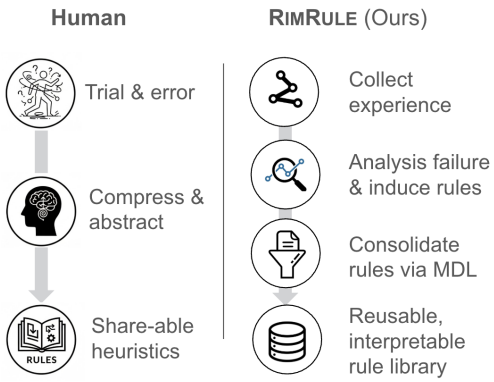


Figure 1: Learning reusable and interpretable rules from experience

We explore a fourth paradigm for adapting LLMs—one that mirrors how humans learn from failure, as illustrated in Fig. 1. Instead of tuning weights or retrieving demonstrations, we induce interpretable rules. At training time, these

[1]*Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics* (Volume 1: Long Papers), pages 34631–34646 July 2-7, 2026 (ACL 2026). Copyright 2026 by the authors.

rules are proposed in response to observed failures. These rules are consolidated using the Minimum Description Length (MDL) principle (Rissanen, 1978; Grunwald, 2007) and stored in a dual natural-language and symbolic representation, enabling both principled compression and reliable retrieval. At inference time, relevant rules are dynamically retrieved and injected into the prompt to improve LLM performance. This constitutes a distinct adaptation paradigm: unlike few-shot prompting, it emphasizes abstraction and compression over raw example replay; unlike finetuning, it produces interpretable artifacts. Because these rules are both understandable to humans and legible to modern LLMs, they can be reused across different LLMs without retraining.

We demonstrate this approach on two tool-use benchmarks—ToolHop (Ye et al., 2025) and BFCL (Yan et al., 2024)—where LLMs must reason over tool descriptions and invoke them correctly. Simple retry mechanisms based on tool error messages, or even the use of advanced long reasoning models, often fall short in reliably correcting systematic failures. Our results highlight several key observations.

- First, inference-time rule injection consistently improves performance, including on queries involving tools that never appear in training.
- Second, learned rules transfer across LLMs in both directions: rules distilled from weaker models improve stronger, long-reasoning LLMs, and rules learned from stronger models can also benefit smaller or less capable ones.
- Third, our method outperforms prompting based adaptation approaches and complements finetuning, offering a distinct and synergistic axis of generalization.

## 2 Method

We introduce RIMRULE[2], a scalable framework that equips LLM-based agents with inference-time intepretable rules distilled from failure traces. Figure 2 illustrates the overall architecture, and Table 1 provides pseudocode.

Instead of learning rules sequentially—an approach that is sensitive to data order and difficult to scale—RIMRULE decouples rule induction into two phases. In the first stage (Section 2.1), candidate rules are generated independently from individual failures, enabling parallel, order-agnostic extraction across large interaction logs. In the second stage (Section 2.2), these candidates are consolidated under an MDL objective, which prunes redundancy and selects a compact, high-utility rule library. This design avoids the path dependence of sequential rule learning while naturally supporting scalable rule discovery and reuse.

### 2.1 Local Rule Generation

#### 2.1.1 Experience Collection

We begin by running a zero-shot LLM-based agent over each sample in the training set and recording its execution trace. When the resulting trace produces an incorrect answer or terminates prematurely due to tool-calling errors, we retain it as experience for rule generation. The collection of such failure traces constitutes the experience signal used for downstream rule induction. Each training instance is represented as a tuple $(x, \mathcal{S}, \tau^-, \tau^*)$, where $x \in \mathcal{X}$ is the user query, $\mathcal{S}$ is the set of available tools, $\tau^-$ is the agent's incorrect execution trace, and $\tau^*$ is the corresponding ground-truth trace.[3]

#### 2.1.2 Rule Generation via EBL

Given a failure trace $\tau^-$ and its ground-truth counterpart $\tau^*$, the rule generator compares the two executions to identify a root-cause *reasoning* failure, excluding downstream propagation effects, and proposes a compact rule $r \in \mathcal{R}_0$ to correct similar errors. Each rule is tagged with an error type $d(r) \in \{\text{dec}, \text{sel}, \text{arg}\}$, corresponding to decomposition, tool selection, or argument construction.

To abstract reusable rules, we follow *Explanation-Based Learning* (EBL): a grounded explanation is generated and then generalized by removing instance-specific details (Mitchell et al., 1986; DeJong and Mooney, 1986). The generator is implemented as an LLM-based function.[4]

Proposed rules are filtered by a *predictive check*, requiring improved performance on the same query when injected, and a *linguistic check* enforcing a clean *if–then* form and bounded length. If a rule

[2] **R**eusable, **I**nterpretable, and **M**DL-guided Rules

[3] In settings without traces but with a reward function or score, the same mechanism can be adapted by proposing multiple candidate rules and selecting those that maximize observed reward on the instance; a full treatment is left to future work.

[4] See appendix for the prompt.

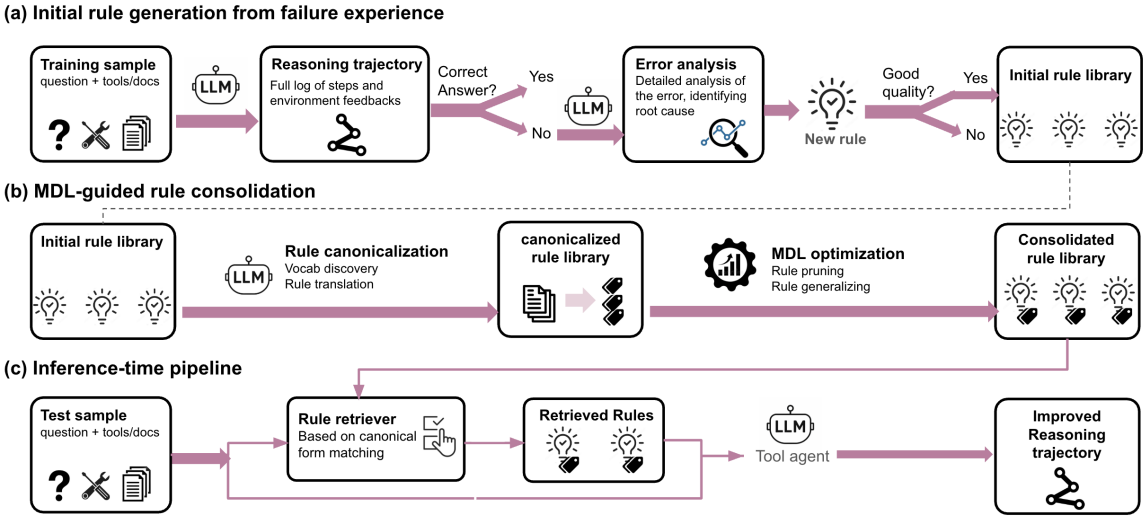


Figure 2: The framework of our MDL-guided rule learning.

yields only partial improvement, the generator is re-invoked to propose additional rules until the trace is corrected or no further atomic fixes are found.

**Algorithm 1** RIMRULE

**Require:** Training failures $D = \{(x_i, \mathcal{S}_i, \tau_i^-, \tau_i^*)\}_{i=1}^n$
**Ensure:** Compact symbolic rule library $\mathcal{R}$

**Stage 1: Rule Generation**
**for all** $(x_i, \mathcal{S}_i, \tau_i^-, \tau_i^*) \in D$ **in parallel do**
  $r_i^{\text{NL}} \leftarrow$ LLM-Propose-Rule$(\cdot)$
  **if** Passes checks **then**
    Add $r_i^{\text{NL}}$ to pool $\mathcal{R}_0$
  **end if**
**end for**
**Symbolic Translation**
Induce vocabulary $\mathcal{V}$ from $\mathcal{R}_0$
Translate $\mathcal{R}_0$ to symbolic rules $\mathcal{R}$
**Stage 2: MDL Consolidation**
**repeat**
  **for all** $r \in \mathcal{R}$ **do**
    Evaluate PRUNE$(r)$ and GENERALIZE$(r)$
    **if** $\Delta L_{\text{MDL}} < 0$ **then**
      Apply best edit
    **end if**
  **end for**
**until** no improvement
**return** $\mathcal{R}$

#### 2.1.3 Symbolic Representation

Natural-language renderings of the same rule often vary in phrasing, hindering both structural matching and consistent description-length computation. We therefore compile each rule into a symbolic representation with fixed fields and closed vocabularies, enabling principled MDL consolidation (Section 2.2.2) and efficient retrieval (Section 2.3).

The symbolic schema consists of five semantic fields: `Domain` (broad topic), `Qualifier` (context), `Action` (prescribed operation), `Strength` (priority), and `ToolCategory` (abstract tool type). Each field draws values from a finite vocabulary.

We construct these vocabularies by prompting an LLM (see Appendix A.1) over batches of natural-language rules and select the most compact vocabulary across randomized orderings. Once fixed, each rule is translated into its symbolic form by assigning valid field values, yielding a deterministic and auditable rule set.

### 2.2 Rule Consolidation

Rule generation typically produces a large set of overlapping and overly specific rules. Consolidation is therefore required to select a compact subset that generalizes well.

We consider two approaches: a prompt-based heuristic that merges rules using an LLM, and a data-driven alternative based on the Minimum Description Length (MDL) principle.

#### 2.2.1 Prompt-Based

A simple consolidation strategy is to prompt an LLM to merge or rewrite candidate rules, as in AutoRule (Wang and Xiong, 2025). While this can reduce redundancy at the textual level, it is heuristic and does not account for how rules affect agent behavior on actual failure cases. This motivates a consolidation objective that explicitly trades off rule complexity against empirical effectiveness.

#### 2.2.2 MDL-Guided

We adopt the *Minimum Description Length* (MDL) principle, which favors models that compress observed data while remaining concise (Rissanen,

1978; Grunwald, 2007). In our setting, MDL provides a principled objective for selecting a compact rule set that corrects as many failures as possible without unnecessary complexity.

**Objective.** Let $H \subseteq \mathcal{R}^{\text{sym}}$ denote a candidate rule library. We minimize

$$\text{MDL}(H) = L(H) + L(D \mid H),$$

where $L(H)$ penalizes rule complexity and $L(D \mid H)$ measures how well the rules correct observed failures.

**Model Cost.** We define a length-based prior over rule libraries,

$$P(H) \propto \exp\Big(-\alpha \sum_{r \in H} \ell(r)\Big),$$

where $\ell(r)$ denotes the **symbolic token length** [5] of rule $r$ and $\alpha > 0$ controls the strength of regularization. This corresponds to a maximum-entropy prior under a constraint on expected total rule length (Jaynes, 1957). Taking the negative log-likelihood yields the model code length

$$L(H) = -\log P(H) = \alpha \sum_{r \in H} \ell(r) + \log Z(\alpha),$$

where $Z(\alpha)$ is constant for a fixed candidate pool and is therefore omitted during optimization.

**Data Cost.** Given failure cases $D = \{(x_i, \mathcal{S}_i, \tau_i^-, \tau_i^*)\}_{i=1}^n$, we define $a_i(H) = 1$ if injecting $H$ corrects failure $i$. Let $k_H = \sum_i a_i(H)$. We model correction outcomes with a Bernoulli likelihood and use the refined MDL plug-in code

$$L(D \mid H) = -\big[k_H \log \hat{p}_H + (n - k_H) \log(1 - \hat{p}_H)\big],$$

where $\hat{p}_H = k_H / n$. This term directly rewards rule sets that correct more failures.

**Greedy Consolidation.** We minimize $\text{MDL}(H)$ starting from the full rule pool and applying local edits that strictly reduce the objective. We consider two operations: *pruning* a rule, and *generalizing* a tool-specific rule to a category-level rule. Edits are accepted only when the reduction in model cost outweighs any increase in data cost. This greedy procedure terminates when no local edit improves the MDL objective.

[5] Symbolic token length provides a canonical measure of rule complexity by abstracting away incidental natural-language phrasing.

In practice, consolidation scales linearly with the size of the candidate rule pool, as each iteration evaluates a constant set of local edits per rule. While this greedy procedure does not guarantee a globally optimal solution and may miss beneficial non-local edits, it follows standard practice in rule learning and MDL-based pruning, where local edits are commonly used to remove redundancy and over-specific rules and are empirically effective in improving generalization (Quinlan, 1987; Fürnkranz, 1997; Grunwald, 2007). Exploring more global or approximate inference strategies for rule consolidation is an interesting direction for future work.

**MDL Behavior.** Although the term $L(D \mid H)$ could be minimized by correcting no failures, this degenerate case is never reached in practice. Consolidation proceeds greedily from a high-performing initial rule set; in this regime, removing a rule that reduces the number of corrected failures sharply increases $L(D \mid H)$, outweighing the modest reduction in model cost. Such edits are therefore rejected.

**Choice of $\alpha$.** We select the regularization strength $\alpha$ by choosing the value that yields the best end-to-end accuracy after consolidation on the failure set. Since each rule is evaluated on failures beyond its originating instance, this selection already reflects cross-sample generalization. Performance is stable across a wide range of $\alpha$, indicating that MDL acts as a robust regularizer.

### 2.3 Rule Retrieval

At inference time, injecting all learned rules is inefficient and noisy, making retrieval necessary to select only rules relevant to the current query and tool context.

#### 2.3.1 Natural language based

A simple baseline retrieves rules directly from their natural-language form by encoding the query and available tools with an off-the-shelf sentence embedding model and selecting the top-$k$ rules by cosine similarity. While easy to implement, this approach relies on generic embeddings that struggle with lexical variability and fine-grained rule semantics. Finetuning a dedicated embedding model introduces additional complexity, including limited training data, the need for ground-truth rule annotations, and reduced scalability.

### 2.3.2 Symbolic-based

To address these limitations, and inspired by prior work on structure-aware retrieval (Munikoti et al., 2023; Li et al., 2023), we perform retrieval over the symbolic representation of rules. By structuring both rules and queries into shared semantic fields, symbolic matching is more efficient, interpretable, and more robust to incidental linguistic variation than retrieval over raw text.

Given a query $x_q$ and available tools $\mathcal{S}_q$, we first convert the state into a symbolic representation $z_q$. Retrieval proceeds in two stages. First, a coarse filter removes inapplicable rules: decomposition rules are always retained, while tool-use rules are filtered based on whether their scope matches the available tools or tool categories. Second, the remaining rules are ranked by semantic similarity between their symbolic fields and those of $z_q$, using a generic embedding model. We retain the top-$k$ rules and inject them into the prompt in natural-language form. Unlike retrieval over raw text, similarity is computed over a fixed symbolic vocabulary, where NL-to-symbolic conversion has already resolved much of the lexical and structural variation that generic embeddings struggle to capture.

Table 1: Number of examples in each dataset split.

| Dataset | Train | Test | |
|---|---|---|---|
| | | Rand | Unseen |
| ToolHop | 392 | 70 | 51 |
| BFCL:Live-Multiple | 735 | 175 | 143 |
| BFCL:Multi-Turn-Base | 90 | 60 | 50 |

## 3 Experiments

### 3.1 Datasets

We evaluate our method on two tool-use benchmarks: **ToolHop**[6] (Ye et al., 2025) and **BFCL**[7] (Yan et al., 2024). ToolHop consists of compositional, multi-turn queries requiring multi-step tool reasoning, while the `live-multiple` subset of BFCL provides a complementary single-step setting with a larger and more diverse tool set, making tool selection more challenging.

For both datasets, we assume access to ground-truth execution traces during training, but no ground-truth rules are provided. To evaluate generalization, we define two splits: **test-rand**, a random in-distribution split, and **test-unseen**, a held-out split containing queries with tools not seen during training. Dataset statistics are summarized in Table 1.

### 3.2 Baselines

We compare RIMRULE against several established adaptation paradigms:

- **Few-shot in-context learning** retrieves top-$k$ training examples based on semantic similarity (Liu et al., 2021).
- **Prompt optimization** with SEE (Cui et al., 2025), which performs global prompt optimization by jointly evolving instructions and demonstrations.
- **Finetuning**, including supervised finetuning (SFT) of open-source models using LoRA (Hu et al., 2022), as well as closed-source models tuned for function calling. For SFT, we construct training data from ground-truth execution traces, training the model to generate correct tool selections and arguments conditioned on the query, available tools, and previous tool-call results.

All methods, including ours, use a **ReAct**-style prompting framework (Yao et al., 2023) for tool-use reasoning, allowing agents to generate intermediate steps and tool calls interactively. We permit agents to retry based on tool feedback or error messages, a common robustness mechanism that nonetheless struggles with systematic reasoning errors. Our method builds on this setup by injecting explicit rules to prevent such failures.

We use short names for evaluated models: `Llama3.2`[8], `Llama4`[9], `GPT-4o`[10], and `O1`[11], where `O1` is a long-reasoning model.

The symbolic representation supports rule consolidation and retrieval, not direct symbolic execution. Because the tool-use tasks we study involve noisy natural-language queries and environments, purely symbolic rule application is brit-

[6] https://hf.co/datasets/bytedance-research/ToolHop, CC BY 4.0 license

[7] https://hf.co/datasets/gorilla-llm/Berkeley-Function-Calling-Leaderboard, Apache license 2.0

[8] meta.llama3-2-3b-instruct-v1-0

[9] meta.llama4-maverick-17b-instruct-v1-0

[10] gpt-4o-2024-11-20

[11] o1-mini-2024-09-12

Table 2: Accuracy (± standard deviation, in %) and number of rules at each learning stage. Rule consolidation improves performance while reducing rule count.

| | ToolHop | | | BFCL | | |
|---|---|---|---|---|---|---|
| | Test-rand | Test-unseen | # Rules | Test-rand | Test-unseen | # Rules |
| Initial | 26.5±1.3 | 35.1±1.6 | 0 | 50.1±0.9 | 45.0±1.0 | 0 |
| + Rule Gen. | 27.6±1.3 | 42.1±1.6 | 72 | 54.8±1.3 | 47.6±1.4 | 151 |
| + Consolid. | **31.1**±1.3 | **43.1**±1.6 | 67 | **56.6**±1.2 | **48.5**±1.4 | 121 |

Table 3: Accuracy (± standard deviation) of LLMs on Toolhop and BFCL, with and without rules. Rules are learned once from Llama3.2 and reused across models.

| Learned from | Applied on | ToolHop | | BFCL | |
|---|---|---|---|---|---|
| | | No Rules | RIMRULE | No Rules | RIMRULE |
| Llama3.2 | Llama3.2 | 26.5±1.3 | **31.1**±1.3 | 50.1±0.9 | **56.6**±1.2 |
| | GPT-4o | 58.1±1.4 | 57.4±1.4 | 71.6±0.8 | **75.6**±1.1 |
| | Llama4 | 73.8±1.2 | **76.7**±1.2 | 75.8±0.8 | **77.5**±1.1 |
| | O1 | 53.2±1.4 | **57.4**±1.4 | 75.6±0.8 | **77.6**±1.1 |
| GPT-4o | Llama3.2 | 26.5±1.3 | **31.3**±1.3 | 50.1±0.9 | **52.4**±1.3 |
| | GPT-4o | 58.1±1.4 | **60.3**±1.4 | 71.6±0.8 | **76.4**±1.1 |
| | Llama4 | 73.8±1.2 | **77.4**±1.2 | 75.8±0.8 | **77.7**±1.0 |
| | O1 | 53.2±1.4 | **56.1**±1.2 | 75.6±0.8 | **77.9**±1.1 |

tle; accordingly, we do not compare against symbolic rule learners designed for clean, structured data(Quinlan, 1990; Clark and Niblett, 1989; Cohen, 1995; Fürnkranz, 1997). Instead, rules are applied in natural-language form to guide LLM reasoning.

## 4 Results and Discussion

### 4.1 Learning Process

Table 2 illustrates the learning trajectory using Llama3.2 as a representative example. Other models exhibit similar trends.

Starting from an empty rule library, we collect failure traces from a zero-shot agent and generate candidate rules, yielding 72 rules on ToolHop and 151 on BFCL[12]. This initial rule set improves accuracy on both test-rand and test-unseen splits.

Applying MDL-guided consolidation further reduces the rule count (by 7% on ToolHop and 20% on BFCL) while improving performance on test-rand and yielding smaller but consistent gains on test-unseen. Overall, the process produces a compact, interpretable rule library that improves both in-distribution and out-of-distribution generalization.

[12]See Appendix for sample rules

### 4.2 Case Study

Table 6 shows a representative failure where the agent directly queries a complex familial relationship, leading to a tool error. RIMRULE identifies the root cause as improper decomposition and induces a rule that enforces stepwise resolution of intermediate entities. The resulting rule is abstract and reusable, and its symbolic form enables reliable retrieval for similar queries. This example illustrates how failure traces are converted into compact rules that correct systematic reasoning errors beyond the original instance.

### 4.3 Re-usability Across LLMs

Humans can share heuristics because they are interpretable across contexts. By encoding adaptation as symbolic, human-readable rules, our method enables knowledge to be reused across LLMs without retraining.

We learn two rule libraries from failure traces produced by `Llama3.2` and `GPT-4o`, and apply them to models of varying size and reasoning strength, evaluating performance on the test-rand split. Table 3 shows consistent gains in both directions: **strong-to-weak** transfer improves smaller models, while **weak-to-strong** transfer benefits strong reasoning models such as `O1` and `Llama4`.

Table 4: Comparison of prompting-based adaptation methods on ToolHop and BFCL.

| | ToolHop | | BFCL | |
|---|---|---|---|---|
| | Test-rand | Test-unseen | Test-rand | Test-unseen |
| Zero-shot | 26.5 ±1.3 | 35.1±1.6 | 50.1 ±0.9 | 45.0±1.0 |
| Few-shot (Liu et al., 2021) | 29.9 ±1.4 | 37.9 ±1.4 | 54.5 ±0.9 | 46.6 ±1.4 |
| SEE (Cui et al., 2025) | 27.6 ±1.5 | 35.9 ±1.5 | 52.2 ±1.0 | 45.5 ±1.0 |
| RIMRULE (Ours) | **31.1**±1.3 | **43.1**±1.6 | **56.6**±1.2 | **48.5**±1.4 |

Table 5: Performance with and without RIMRULE for finetuned models.

| | ToolHop | | BFCL | |
|---|---|---|---|---|
| | Test-rand | Test-unseen | Test-rand | Test-unseen |
| SFT (`Llama3.2`) | 43.8 ±1.4 | 38.5 ±1.5 | 65.0 ±0.9 | 58.7 ±1.4 |
| + RIMRULE | **50.0** ±1.4 | **45.1** ±1.6 | **68.6** ±1.2 | **62.7** ±1.3 |
| Function Calling (`GPT4o`) | 63.8 ±1.4 | 83.0 ±1.2 | 79.6 ±0.7 | 77.2 ±0.8 |
| + RIMRULE | **66.1** ±1.4 | **85.4** ±1.1 | **81.7** ±1.0 | **79.8** ±1.0 |

Table 6: Case study illustrating failure-driven rule induction and symbolic representation.

| | |
|---|---|
| **Query** | How many letters (excluding the first and last) are there in the first name of Viacheslav I of Kiev's maternal grandfather? |
| **Failure** | The agent directly queries *Viacheslav I of Kiev's maternal grandfather*. The tool returns an error: *"no data found"*. |
| **Proposed Rule** | If the user query involves identifying a specific familial relationship (e.g., maternal grandfather), then decompose the task by first resolving intermediate relationships (e.g., mother, father) sequentially. |
| **Symbolic Form** | `if domain=FAMILIAL_RELATIONSHIP` **or** `tool_category=GENEALOGY_QUERY, then action=[DECOMPOSE_QUERY, RESOLVE_INTERMEDIATE_ENTITY, SEQUENCE_SUBTASKS] strength=MANDATORY` |

These results suggest that symbolic rules capture transferable failure patterns not resolved by scale alone.

### 4.4 Performance on Small Dataset

We evaluate RIMRULE in a low-resource setting on the `multi_turn_base` split of BFCL using only 90 training samples. Despite limited supervision, the method learns just four rules (Appendix A.2) yet achieves substantial gains: accuracy improves from 55.2% to 62.1% on test-rand and from 46.0% to 60.0% on test-unseen. These results highlight the sample efficiency and practical applicability of rule-based adaptation.

Table 7: Test-rand accuracy (± standard deviation, in %) comparing retrieval strategies.

| Method | ToolHop | BFCL |
|---|---|---|
| Nat. Lang. based | 29.4±1.3 | 54.1±1.0 |
| Symbolic guided | **31.1**±1.3 | **56.6**±1.2 |

### 4.5 Outperforming Prompting-based Methods

We compare RIMRULE against prompting-based adaptation methods, including ReAct prompting (Yao et al., 2023), few-shot in-context learning (Liu et al., 2021), and prompt optimization with SEE (Cui et al., 2025). As shown in Table 4, RIMRULE consistently outperforms all prompting baselines on both ToolHop and BFCL, across test-rand and test-unseen splits, demonstrating stronger generalization than fixed-prompt approaches.

### 4.6 Complementing Finetuned Models

As shown in Table 5, adding RIMRULE to finetuned models yields consistent gains, especially on the test-unseen split, indicating that symbolic rules correct residual systematic errors left by supervised training. We observe similar improvements when applying RIMRULE to models with native function-calling capabilities (Table 5), showing that rule-based adaptation operates along an orthogonal axis of generalization.

### 4.7 Ablation Studies

**Impact of retrieval method.** Table 7 shows that symbolic retrieval consistently outperforms

Table 8: Test-rand accuracy (± standard deviation, in %) comparing consolidation strategies.

| Method | ToolHop | BFCL |
|---|---|---|
| Prompt-based | 27.5±1.4 | 52.1±0.9 |
| MDL-guided | **31.1**±1.3 | **56.6**±1.2 |

retrieval based on raw natural-language similarity on both datasets.

**Impact of consolidation method.** As shown in Table 8, MDL-guided consolidation significantly outperforms prompt-based rule merging. Together, these results indicate that both symbolic retrieval and MDL-based consolidation are essential to our approach.

## 5 Related Work

**Rule Learning.** Recent work studies rule learning with LLMs via prompting, extracting rules from demonstrations (Gao and Das, 2024), graphs (Chen et al., 2024), or traces (Zhang et al., 2024) to guide generation (Zhou et al., 2024; Wang et al., 2024b). Other approaches use rules at training time for synthetic supervision (Morishita et al., 2024), reward modeling (Wang and Xiong, 2025), or distillation (Sadeq et al., 2025), or store rules in memory (Wang et al., 2024c). In contrast to these largely static or training-time uses, we learn and consolidate rules from failures for dynamic inference-time reuse.

**Tool-use Agents.** Tool-use agents often rely on retriever–generator pipelines or finetuning to select and call tools (Qin et al., 2023; Patil et al., 2023), or train models to use tools via self-supervision or agent tuning (Schick et al., 2023). Recent work further explores reinforcement learning and agent-centric optimization for tool use (Le et al., 2025; Zhang et al., 2025), as well as generative approaches that tokenize tools for unified retrieval and calling (Hao et al., 2023; Wang et al., 2024a). While effective, these approaches primarily encode adaptation in model parameters. We instead externalize adaptation as a compact, interpretable rule library, enabling modular reuse across LLMs without retraining.

## 6 Conclusion

We propose RIMRULE, an approach that adapts LLMs by distilling compact, interpretable rules from failure traces and applying them at inference time. Without modifying model weights, our method improves both in- and out-of-distribution performance, outperforms prompting-based methods, and complements finetuned models. These results position inference-time rule learning as a practical path toward modular and interpretable LLM adaptation.

## 7 Limitations

Our approach assumes access to failure signals during training, such as execution traces or reliable performance feedback, which may not be available in all deployment settings. Rule consolidation is performed using a greedy MDL-based procedure; while efficient and effective at the scales studied, it does not guarantee a globally optimal rule set, and alternative optimization methods deserve future study. In addition, the symbolic representation relies on a manually designed schema intended for noisy natural-language tool-use tasks; other domains may benefit from different symbolic abstractions. Finally, the effectiveness of learned rules depends on the quality and diversity of observed failures, and gains may be smaller in domains with sparse or highly idiosyncratic errors.

**Potential Risks.** As with any adaptive system, learned rules could overgeneralize or reflect biases present in observed failures if applied outside their intended scope. We mitigate such risks by keeping rules interpretable and applied at inference time, enabling auditing, revision, or removal. Future work may further explore safeguards for monitoring and validating rule usage in high-stakes settings.

## References


Jinze Bai, Shuai Bai, Yunfei Chu, Zeyu Cui, Kai Dang, Xiaodong Deng, Yang Fan, Wenbin Ge, Yu Han, Fei Huang, and 1 others. 2023. Qwen technical report. *arXiv preprint arXiv:2309.16609*.

Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, and 1 others. 2020. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901.

Zhongwu Chen, Chengjin Xu, Dingmin Wang, Zhen Huang, Yong Dou, Xuhui Jiang, and Jian Guo. 2024. Rulerag: Rule-guided retrieval-augmented generation with language models for question answering. *arXiv preprint arXiv:2410.22353*.

Peter Clark and Tim Niblett. 1989. The cn2 induction algorithm. *Machine Learning*, 3(4):261–283.

William W. Cohen. 1995. Fast effective rule induction. In *Proceedings of the 12th International Conference on Machine Learning (ICML-95)*, pages 115–123, San Francisco, CA. Morgan Kaufmann.

Wendi Cui, Jiaxin Zhang, Zhuohang Li, Hao Sun, Damien Lopez, Kamalika Das, Bradley A Malin, and Sricharan Kumar. 2025. See: Strategic exploration and exploitation for cohesive in-context prompt optimization. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 29575–29627.

Gerald DeJong and Raymond Mooney. 1986. Explanation-based learning: An alternative view. *Machine learning*, 1:145–176.

Johannes Fürnkranz. 1997. Pruning algorithms for rule learning. *Machine Learning*, 27(2–3):139–171.

Xiang Gao and Kamalika Das. 2024. Customizing language model responses with contrastive in-context learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pages 18039–18046.

Gerd Gigerenzer and Wolfgang Gaissmaier. 2011. Heuristic decision making. *Annual Review of Psychology*, 62:451–482.

Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, and 1 others. 2024. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*.

Peter D. Grunwald. 2007. *The Minimum Description Length Principle*. MIT Press, Cambridge, MA.

Shibo Hao, Tianyang Liu, Zhen Wang, and Zhiting Hu. 2023. Toolkengpt: Augmenting frozen language models with massive tools via tool embeddings. *arXiv preprint*.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. Lora: Low-rank adaptation of large language models. In *ICLR*.

E. T. Jaynes. 1957. Information theory and statistical mechanics. *Physical Review*, 106(4):620–630.

David A. Kolb. 1984. *Experiential Learning: Experience as the Source of Learning and Development*. Prentice Hall, Englewood Cliffs, NJ.

Quy Minh Le, Minh Sao Khue Luu, Khanh-Tung Tran, Duc-Hai Nguyen, Hoang-Quoc-Viet Pham, Quan Le, Hoang Thanh Lam, and Hoang D. Nguyen. 2025. Toolbrain: A flexible reinforcement learning framework for agentic tools. Accessed 2026-01-xx.

Xinze Li, Zhenghao Liu, Chenyan Xiong, Shi Yu, Yu Gu, Zhiyuan Liu, and Ge Yu. 2023. Structure-aware language model pretraining improves dense retrieval on structured data. In *Findings of the Association for Computational Linguistics: ACL 2023*, pages 11560–11574.

Jiachang Liu, Dinghan Shen, Yizhe Zhang, Bill Dolan, Lawrence Carin, and Weizhu Chen. 2021. What makes good in-context examples for gpt-3? *arXiv preprint arXiv:2101.06804*.

Janet Metcalfe. 2017. Learning from errors. *Annual Review of Psychology*, 68:465–489.

Tom M Mitchell, Richard M Keller, and Miriam A Kedar-Cabelli. 1986. Explanation-based generalization: A unifying view. *Machine Learning*, 1(1):47–80.

Terufumi Morishita, Gaku Morio, Atsuki Yamaguchi, and Yasuhiro Sogawa. 2024. Enhancing reasoning capabilities of llms via principled synthetic logic corpus. *Advances in Neural Information Processing Systems*, 37:73572–73604.

Sai Munikoti, Anurag Acharya, Sridevi Wagle, and Sameera Horawalavithana. 2023. Atlantic: Structure-aware retrieval-augmented language model for interdisciplinary science.

Shishir G. Patil, Rohan Taori, and 1 others. 2023. Gorilla: Large language model connected with massive apis.

Reid Pryzant, Dan Iter, Jerry Li, Yin Tat Lee, Chenguang Zhu, and Michael Zeng. 2023. Automatic prompt optimization with" gradient descent" and beam search. *arXiv preprint arXiv:2305.03495*.

Shihan Qin, Chen Zhang, and et al. 2023. Toolllm: Facilitating large language models to master 16000+ real-world apis. *arXiv preprint arXiv:2307.16789*.

J. Ross Quinlan. 1987. Simplifying decision trees. *International Journal of Man-Machine Studies*, 27(3):221–234.

J. Ross Quinlan. 1990. Learning logical definitions from relations. *Machine Learning*, 5(3):239–266.

Jorma Rissanen. 1978. Modeling by shortest data description. *Automatica*, 14(5):465–471.

Eleanor Rosch. 1978. Principles of categorization. In Eleanor Rosch and Barbara B. Lloyd, editors, *Cognition and Categorization*, pages 27–48. Lawrence Erlbaum Associates, Hillsdale, NJ.

Nafis Sadeq, Xin Xu, Zhouhang Xie, Julian McAuley, Byungkyu Kang, Prarit Lamba, and Xiang Gao. 2025. Improving in-context learning with reasoning distillation. *arXiv preprint arXiv:2504.10647*.

Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Gemini Team and Others. 2023. Gemini: A family of highly capable multimodal models. *arXiv preprint*.

Mudit Verma, Siddhant Bhambri, and Subbarao Kambhampati. 2024. On the brittle foundations of react prompting for agentic large language models. *arXiv preprint arXiv:2405.13966*.

Renxi Wang, Xudong Han, Lei Ji, Shu Wang, Timothy Baldwin, and Haonan Li. 2024a. Toolgen: Unified tool retrieval and calling via generation. *arXiv preprint*.

Siyuan Wang, Zhongyu Wei, Yejin Choi, and Xiang Ren. 2024b. Can llms reason with rules? logic scaffolding for stress-testing and improving llms. *arXiv preprint arXiv:2402.11442*.

Siyuan Wang, Zhongyu Wei, Yejin Choi, and Xiang Ren. 2024c. Symbolic working memory enhances language models for complex rule application. *arXiv preprint arXiv:2408.13654*.

Tevin Wang and Chenyan Xiong. 2025. Autorule: Reasoning chain-of-thought extracted rule-based rewards improve preference learning.

Jerry Wei, Jason Wei, Yi Tay, Dustin Tran, Albert Webson, Yifeng Lu, Xinyun Chen, Hanxiao Liu, Da Huang, Denny Zhou, and 1 others. 2023. Larger language models do in-context learning differently. *arXiv preprint arXiv:2303.03846*.

Fanjia Yan, Huanzhi Mao, Charlie Cheng-Jie Ji, Tianjun Zhang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. 2024. Berkeley function calling leaderboard. https://gorilla.cs.berkeley.edu/blogs/8_berkeley_function_calling_leaderboard.html.

Enneng Yang, Li Shen, Guibing Guo, Xingwei Wang, Xiaochun Cao, Jie Zhang, and Dacheng Tao. 2024. Model merging in llms, mllms, and beyond: Methods, theories, applications and opportunities. *arXiv preprint arXiv:2408.07666*.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*.

Junjie Ye, Zhengyin Du, Xuesong Yao, Weijian Lin, Yufei Xu, Zehui Chen, Zaiyuan Wang, Sining Zhu, Zhiheng Xi, Siyu Yuan, and 1 others. 2025. Toolhop: A query-driven benchmark for evaluating large language models in multi-hop tool use. *arXiv preprint arXiv:2501.02506*.

Yabo Zhang, Yihan Zeng, Qingyun Li, Zhen Hu, Kavin Han, and Wangmeng Zuo. 2025. Tool-r1: Sample-efficient reinforcement learning for agentic tool use. https://arxiv.org/abs/2509.12867. Accessed 2026-01-xx.

Yudi Zhang, Pei Xiao, Lu Wang, Chaoyun Zhang, Meng Fang, Yali Du, Yevgeniy Puzyrev, Randolph Yao, Si Qin, Qingwei Lin, and 1 others. 2024. Ruag: Learned-rule-augmented generation for large language models. *arXiv preprint arXiv:2411.03349*.

Pei Zhou, Jay Pujara, Xiang Ren, Xinyun Chen, Heng-Tze Cheng, Quoc V Le, Ed Chi, Denny Zhou, Swaroop Mishra, and Huaixiu Steven Zheng. 2024. Self-discover: Large language models self-compose reasoning structures. *Advances in Neural Information Processing Systems*, 37:126032–126058.

# A Appendix

## A.1 Prompts

This section includes the prompts used throughout our pipeline: for generating rules from failure traces, discovering and updating the classification vocabulary, and translating rules into canonical form. Prompts are shown as used, with placeholders for query, tools, and traces.

```
You are a senior AI systems instructor
    tasked with helping a tool-using
    language model, referred to as the
    Tool Agent, improve its ability to
    use tools accurately, efficiently,
    and reliably, even with new tools
    and unseen queries that are
    structurally and/or semantically
    similar to previously seen
    examples. Your role is to review
    incorrect tool usage traces
    produced by the Tool Agent, compare
    them against the correct
    (groundtruth) traces, and analyze
    any embedded error messages to
    reason about the error.

An incorrect trace occurs when a
    query's execution by the Tool Agent
    doesn't match the groundtruth
    answer. Incorrect traces are either
    complete (finished but incorrect)
    or incomplete due to unrecoverable
    execution errors. You will be
    provided with the Tool Agent's full
    execution trace, including embedded
    error messages.

There are two types of errors:
i. Reasoning errors (to analyze and
    generate rules for)
ii. Propagation errors (to ignore, as
    they result from prior mistakes)

Steps may contain:
(i) no errors, (ii) only propagation
    errors, or (iii) one or more
    reasoning errors.

Your goal is to extract generalizable,
    tool-schema-aware, and
    context-sensitive rules that can
    guide future tool usage-even with
    unseen queries and schemas. Think
    like a meta-cognitive teacher:
    diagnose root causes and articulate
    precise, reusable abstractions.

You will now be given:
1. The user query [INSERT USER QUERY]
2. The available tools [INSERT TOOL
    SCHEMA JSON]
3. The incorrect trace [INSERT TOOL
    AGENT TRACE]
4. The groundtruth trace [INSERT
    GROUNDTRUTH TRACE]

Your task is to identify reasoning
    errors by comparing the traces.
Check:
- Were subtasks identified correctly?
- Were the correct tools selected?
- Were the arguments constructed
    correctly?

Guidelines for rule generation:
- Only generate rules for reasoning
    errors
- Identify root causes, not superficial
    fixes
- Rules must be generalizable and
    schema-aware
- Rules must not contain query-specific
    or tool-specific tokens
- Each rule must be atomic and
    classifiable as:
  a. decomposition error
  b. tool selection error
  c. tool arguments error
- If multiple reasoning failures occur,
    generate multiple rules unless
    clarity is preserved

Rules must be symbolic, composable, and
    conform to a standard form (e.g.,
    condition => action).

Output format:
First, write a brief analysis
    explaining the root cause and error
    type.
Then output one rule only, using this
    JSON structure:
{
  "new_rule": "<generalized if-then
      rule>",
  "error_type": "<decomposition error /
      tool selection error / tool
      arguments error>"
}
```

Listing 1: Prompt used to generate reasoning rules from failure traces. Placeholders like user query and tool schemas are injected at runtime.

```
You are an expert rule classification
    specialist.
Your task is to CREATE the initial
    vocabulary for rule classification
    based on the provided field
    definitions.
Analyze the rules carefully and create
    comprehensive categories that cover
    all rule types.
Use only uppercase letters, numbers,
    and underscores for token names.
Return only valid JSON format.

Create initial vocabulary for rule
    classification based on these
    definitions:

FIELD DEFINITIONS:

DOMAIN (enum): Broad topical domain,
```

```
    inferred jointly from rule content
    + tool context + query context.

QUALIFIER (enum): Fine-grained
    situational tags that are more
    specific than domain.

ACTION (enum): The action(s) the rule
    prescribes, from a closed set.

STRENGTH (enum): Rule action priority
    (must, may). Same
    scope/domain/qualifier, higher
    priority should be retrieved rather
    than low priority.
Examples: MANDATORY, RECOMMENDED,
    OPTIONAL

TOOL_CATEGORY (enum): Tool functional
    category based on tool capabilities
    and usage patterns.
This should represent broad tool types
    that can group similar tools
    together.
Examples: DATA_PROCESSING,
    SEARCH_ENGINE, COMPUTATION,
    TEXT_PROCESSING...

Rules to analyze:
[INSERT_BULLETS_HERE]

Create a comprehensive vocabulary
    covering all rule types in the
    above rules.
Use only uppercase letters, numbers,
    and underscores for category names.

Return JSON:
{
  "vocab_version": "v1",
  "domain": [ ... ],
  "qualifier": [ ... ],
  "action": [ ... ],
  "strength": [ ... ],
  "tool_category": [ ... ]
}
```

Listing 2: Prompt used to induce the rule classification vocabulary from a batch of rules. Placeholders are filled dynamically.

```
You are an expert rule classification
    specialist.
Your task is to UPDATE and EXPAND an
    existing vocabulary based on new
    rules.
Review the current vocabulary, identify
    missing categories from new rules,
    and return the complete updated
    vocabulary.
Use only uppercase letters, numbers,
    and underscores for token names.
Avoid duplicates and maintain
    consistency with existing
    categories.
Return only valid JSON format.

Update and expand vocabulary for rule
    classification.

FIELD DEFINITIONS:

DOMAIN (enum): Broad topical domain,
    inferred jointly from rule content
    + tool context + query context.

QUALIFIER (enum): Fine-grained
    situational tags that are more
    specific than domain.

ACTION (enum): The action(s) the rule
    prescribes, from a closed set.

STRENGTH (enum): Rule action priority
    (must, may). Same
    scope/domain/qualifier, higher
    priority should be retrieved rather
    than low priority.

TOOL_CATEGORY (enum): Tool functional
    category based on tool capabilities
    and usage patterns.

Current accumulated vocabulary:
DOMAIN: [INSERT CURRENT DOMAINS OR
    'None yet']
QUALIFIER: [INSERT CURRENT QUALIFIERS
    OR 'None yet']
ACTION: [INSERT CURRENT ACTIONS OR
    'None yet']
STRENGTH: [INSERT CURRENT STRENGTHS OR
    'None yet']
TOOL_CATEGORY: [INSERT CURRENT
    TOOL_CATEGORIES OR 'None yet']

New rules to analyze:
[INSERT NEW RULE BULLETS HERE]

Instructions:
1. Review the current vocabulary above
2. Analyze the new rules to identify
    any missing categories
3. Add new categories if needed, but
    avoid duplicates
4. Keep existing categories that are
    still relevant
5. Return the complete updated
    vocabulary

Return JSON with all categories
    (existing + new):
{
  "domain": ["existing_domains",
      "new_domains_if_any"],
  "qualifier": ["existing_qualifiers",
      "new_qualifiers_if_any"],
  "action": ["existing_actions",
      "new_actions_if_any"],
  "strength": ["existing_strengths",
      "new_strengths_if_any"],
  "tool_category":
      ["existing_tool_categories",
      "new_tool_categories_if_any"]
}
```

Listing 3: Prompt used to update and expand the rule classification vocabulary using newly generated rules.

```
You are an expert rule classification
    specialist.
Your task is to classify individual
    rules using the provided vocabulary.
Analyze the rule content, tool context,
    and query context to determine the
    most appropriate classification.
Choose only from the given vocabulary
    options.
Return only valid JSON format.

Vocab enumerations:
- domain: [INSERT domain list]
- qualifier: [INSERT qualifier list]
- action: [INSERT action list]
- strength: [INSERT strength list]
- tool_category: [INSERT tool_category
    list]

Rule to classify (analyze all fields
    for context):
[INSERT RULE OBJECT AS JSON]

Classification Guidelines:

DOMAIN: Infer from rule content + tool
    context + query context. Choose the
    broadest applicable domain that
    covers the rule's topic.

QUALIFIER: Select fine-grained
    situational tags that are more
    specific than domain. You can
    select multiple qualifiers if the
    rule applies to multiple
    situations. Focus on conditions,
    constraints, or specific contexts
    mentioned in the rule.

ACTION: Identify the specific action(s)
    the rule prescribes. Choose from
    the closed set of available
    actions. You can select multiple
    actions if the rule prescribes
    multiple steps.

STRENGTH: Determine rule priority
    (must, may). MANDATORY = must
    follow, RECOMMENDED = should
    follow, OPTIONAL = may follow.
    Consider the rule's language and
    context to determine priority.

TOOL_CATEGORY: Identify the functional
    category of the tool(s) used in
    this rule. Choose from the
    available tool categories based on
    the tool's capabilities and usage
    pattern. This should represent the
    broad functional type of the tool
    (e.g., DATA_PROCESSING,
    SEARCH_ENGINE, COMPUTATION).

Return ONLY valid JSON matching this
    schema:
{
  "_id": 0,
  "domain": "FAMILIAL_RELATIONSHIPS",
  "qualifier": ["SEQUENTIAL",
      "MULTI_STEP"],
  "action": ["DECOMPOSE", "VALIDATE"],
  "strength": "MANDATORY",
  "tool_category": "DATA_PROCESSING"
}
```

Listing 4: Prompt used to classify individual rules using a fixed vocabulary. Vocabulary values and rule content are injected at runtime.

### A.2 Sample Rules

Below, we present a representative subset of learned rules. Each rule is shown in natural language along with its corresponding symbolic representation.

```
1. If the user query involves
    identifying a complex relationship
    (e.g., step-relative) and the
    available tools do not directly
    support the requested relationship
    type, then decompose the query into
    intermediate subtasks that
    progressively resolve the
    relationship through simpler,
    directly supported relationship
    types.
if (domain=RELATIONSHIP_RESOLUTION and
    qualifier=[RELATIONSHIP_CHAIN
_TRAVERSAL,
    INTERMEDIATE_ENTITY_IDENTIFICATION])
    then (action=[DECOMPOSE_QUERY,
    RESOLVE_INTERMEDIATE_ENTITY]) with
    strength=MANDATORY

2. If a query involves determining a
    property of an object derived from
    intermediate steps (e.g., counting,
    analyzing, or transforming an
    attribute of an entity), then the
    decomposition must explicitly
    include a subtask to apply the
    appropriate process or
    transformation to the derived
    attribute.
if (domain=QUERY_DECOMPOSITION and
    qualifier=[MULTI_STEP_REASONING,
    DERIVATIVE_ENTITY]) then
    (action=[DECOMPOSE_QUERY,
    TRANSFORM_INPUT]) with
    strength=MANDATORY

3. If a query fails due to insufficient
    or incomplete data returned by a
    tool, then refine the input
    arguments by including optional
    parameters that provide additional
    context (e.g., variants of names,
    time periods, regions, or other
    applicable constraints) to improve
    data retrieval accuracy.
if (domain=TOOL_ARGUMENT_VALIDATION and
    qualifier=[INCOMPLETE_DATA
_REFINEMENT]) then
    (action=[REFINE_INPUT_ARGUMENTS,
    HANDLE_INCOMPLETE_DATA]) with
    strength=RECOMMENDED
```

```
4. If a query involves determining an
    attribute (e.g., date, location,
    event) of an entity that is related
    to another entity (e.g., family
    member), then decompose the query
    to first identify the related
    entity before attempting to
    determine its attribute.
if (domain=QUERY_DECOMPOSITION and
    qualifier=[RELATIONSHIP_CHAIN_
TRAVERSAL,
    INTERMEDIATE_ENTITY_IDENTIFICATION])
    then (action=[DECOMPOSE_QUERY,
    RESOLVE_INTERMEDIATE_ENTITY]) with
    strength=MANDATORY

5. If the subtask requires extracting a
    specific subset of information
    (e.g., first name from a full
    name), then construct arguments for
    the tool to restrict the output to
    match the requested subset by
    enabling relevant optional
    parameters explicitly, while
    leaving unrelated parameters at
    their defaults.
if (domain=NAME_PROCESSING and
    qualifier=[NAME_COMPONENT_EXTRACTION])
    then (action=[CONSTRUCT_ARGUMENTS,
    REFINE_INPUT_ARGUMENTS]) with
    strength=MANDATORY

6. If the user query involves
    identifying information about a
    specific familial relationship
    across multiple generations (e.g.,
    paternal grandfather), then
    decompose the query such that the
    familial relationship is resolved
    directly using tools designed to
    retrieve genealogical data, without
    redundantly extracting unrelated
    immediate relationships.
if (domain=RELATIONSHIP_RESOLUTION and
    qualifier=[FAMILIAL_RELATIONSHIP,
    RELATIONSHIP_CHAIN_TRAVERSAL]) then
    (action=[DECOMPOSE_QUERY,
    IDENTIFY_RELATIONSHIP]) with
    strength=MANDATORY

7. If a query involves determining the
    ancestry of a person based on a
    specific familial relationship
    (e.g., maternal grandfather), then
    decompose the task into subtasks
    that sequentially query each
    familial link (e.g., mother,
    mother's father) and ensure
    intermediate results are used to
    refine subsequent subtasks.
if (domain=HIERARCHICAL_RELATIONSHIPS
    and
    qualifier=[FAMILIAL_RELATIONSHIP,
    RELATIONSHIP_CHAIN_TRAVERSAL,
    INTERMEDIATE_ENTITY_IDENTIFICATION])
    then (action=[DECOMPOSE_QUERY,
    SEQUENCE_SUBTASKS,
    RESOLVE_INTERMEDIATE_ENTITY]) with
    strength=MANDATORY

8. If a query involves identifying an
    entity indirectly related to
    another (e.g., the founder of a
    political party identified in a
    previous step), then ensure that
    intermediate subtasks chain outputs
    logically by querying the
    relationship explicitly rather than
    assuming associations provided in
    the query.
if (domain=INDIRECT_ENTITY
_IDENTIFICATION and
    qualifier=[RELATIONSHIP_CHAIN
_TRAVERSAL,
    INTERMEDIATE_ENTITY_IDENTIFICATION])
    then (action=[SEQUENCE_SUBTASKS,
    RESOLVE_INTERMEDIATE_ENTITY]) with
    strength=MANDATORY

9. If a query involves retrieving
    information about an individual's
    relative, the decomposition must
    include a subtask to explicitly
    identify the relative before
    retrieving specific information
    about them.
if (domain=RELATIONSHIP_RESOLUTION and
    qualifier=[FAMILIAL_RELATIONSHIP,
    RELATIONSHIP_CHAIN_TRAVERSAL]) then
    (action=[DECOMPOSE_QUERY,
    RESOLVE_INTERMEDIATE_ENTITY]) with
    strength=MANDATORY

10. If the subtask involves identifying
    a familial or social relationship
    between entities, then select a
    tool designed to query
    relationships, and do not select a
    tool designed for extracting name
    components or parsing names.
if (domain=RELATIONSHIP_RESOLUTION and
    qualifier=[FAMILIAL_RELATIONSHIP,
    RELATIONSHIP_CHAIN_TRAVERSAL]) then
    (action=[MATCH_TOOL_TO_SUBTASK])
    with strength=MANDATORY
```

Listing 5: Sample rules learned from the ToolHop dataset

```
1. If a reasoning process identifies a
    subtask requiring tool-based
    execution, then ensure the tool
    name generated in the output
    matches exactly with one of the
    available tools in the schema. Use
    schema validation to confirm that
    the generated tool name exists
    before finalizing the response.
if (domain=TRANSPORTATION and
    qualifier=[SCHEMA_VALIDATION,
    TOOL_SELECTION_ERROR]) then
    (action=[CONFIRM_SCHEMA_COMPLIANCE,
    SELECT_TOOL_BASED_ON_QUERY]) with
    strength=MANDATORY

2. IF the query involves retrieving
    general information about an
    entity, concept, or event that is
```

```
    not explicitly tied to recent
    updates or temporal relevance, THEN
    select a tool designed for general
    web searches instead of a tool
    specialized for retrieving recent
    news or updates.
if (domain=GENERAL_INFORMATION

_RETRIEVAL and
    qualifier=[TOOL_SELECTION_ERROR])
    then
    (action=[SELECT_TOOL_BASED_ON_QUERY])
    with strength=MANDATORY

3. If the subtask requires checking
    availability or stock levels for
    specific product attributes (e.g.,
    sizes, stock count), then select a
    tool whose schema explicitly
    supports inventory-related
    operations, rather than using a
    tool designed for general product
    details.
if (domain=INVENTORY_MANAGEMENT and
    qualifier=[TOOL_SELECTION_ERROR])
    then
    (action=[SELECT_TOOL_BASED_ON_QUERY])
    with strength=MANDATORY

4. If the subtask involves checking the
    availability of a specific
    attribute (e.g., size, color, or
    stock) for a product, then select a
    tool whose schema explicitly
    includes functionality for querying
    availability of attributes rather
    than retrieving general product
    details.
if (domain=INVENTORY_MANAGEMENT and
    qualifier=[TOOL_SELECTION_ERROR])
    then
    (action=[SELECT_TOOL_BASED_ON_QUERY])
    with strength=MANDATORY

5. If a user query explicitly provides
    a value for a parameter, regardless
    of whether the parameter has a
    default value in the schema,
    prioritize the user-provided value
    over the default value when
    constructing arguments.
if (domain=DATABASE_MANAGEMENT and
    qualifier=[USER_QUERY_VALUE_PRIORITY,
    DEFAULT_VALUE_ASSIGNMENT]) then
    (action=[PRIORITIZE_USER_VALUE,
    CONSTRUCT_TOOL_ARGUMENTS]) with
    strength=MANDATORY

6. If the user query explicitly
    requests understanding,
    explanation, or help about a
    functionality, then select a tool
    designed to provide informational
    assistance rather than one intended
    for operational execution.
if
    (domain=GENERAL_INFORMATION_RETRIEVAL
    and
    qualifier=[TOOL_SELECTION_ERROR])
    then
    (action=[SELECT_TOOL_BASED_ON_QUERY])
    with strength=MANDATORY

7. If the intended action is to
    retrieve information based on
    user-specified filters or
    constraints, then select a tool
    that explicitly supports filtering
    or constraint-based retrieval in
    its schema description. Ensure the
    subtask aligns directly with the
    tool's primary function as
    specified in its schema.
if
    (domain=GENERAL_INFORMATION_RETRIEVAL
    and
    qualifier=[TOOL_SELECTION_ERROR])
    then
    (action=[SELECT_TOOL_BASED_ON_QUERY])
    with strength=MANDATORY

8. If a subtask involves identifying
    available options or exploring
    possible choices for a resource
    (e.g., accommodations, flights,
    attractions), then select a tool
    designed for searching or browsing
    those resources rather than a tool
    designed for finalizing or
    reserving them.
if
    (domain=GENERAL_INFORMATION_RETRIEVAL
    and
    qualifier=[TOOL_SELECTION_ERROR])
    then
    (action=[SELECT_TOOL_BASED_ON_QUERY])
    with strength=MANDATORY

9. If the user query specifies a
    constraint related to an entity
    (e.g., actor name, director name,
    genre, etc.), then construct the
    corresponding parameter value by
    directly extracting the entity
    mentioned in the query, instead of
    using default or placeholder values.
if
    (domain=GENERAL_INFORMATION_RETRIEVAL
    and qualifier=[REQUIRED_PARAMETER

_FORMATTING,
    USER_QUERY_VALUE_PRIORITY]) then
    (action=[CONSTRUCT_TOOL_ARGUMENTS,
    PRIORITIZE_USER_VALUE]) with
    strength=MANDATORY

10. If a schema explicitly limits valid
    values for an input parameter to a
    predefined set, then when
    constructing arguments, map user
    preferences to the closest valid
    value within this set, or default
    to a neutral option (e.g., no
    filtering) when the schema
    disallows explicit preferences.
if
    (domain=GENERAL_INFORMATION_RETRIEVAL
    and qualifier=[SCHEMA_VALIDATION,
    DEFAULT_VALUE_ASSIGNMENT]) then
    (action=[CONSTRUCT_TOOL_ARGUMENTS,
```

```
    ASSIGN_DEFAULT_VALUE]) with
    strength=MANDATORY
```

Listing 6: Sample rules learned from the BFCL: Live-Multiple dataset

```
1. If a subtask involves initiating a
    system or process with
    preconditions, then identify and
    include all required preconditions
    (e.g., state validations,
    sequential actions) in the task
    decomposition, ensuring that they
    are executed in the correct order
    prior to invoking the main action.
    For example, if starting a process
    requires preconditions A, B, and C,
    verify and satisfy A, B, and C
    sequentially before initiating the
    process.
if (domain=PROCESS_MANAGEMENT and
    qualifier=[PRECONDITION_VALIDATION,
    TASK_SEQUENCE_VALIDATION]) then
    (action=[DECOMPOSE_TASK,
    VALIDATE_PRECONDITIONS,
    ENSURE_SEQUENCE]) with
    strength=MANDATORY

2. If a user query involves an action
    that requires specific
    prerequisites to be met (e.g., a
    dependency between tasks or a state
    requirement), decompose the query
    into subtasks that explicitly
    address the prerequisites first.
    Ensure each prerequisite subtask is
    executed and validated before
    proceeding with the dependent
    action.
if (domain=PROCESS_MANAGEMENT and
    qualifier=[PREREQUISITE_TASK,
    DEPENDENCY_MANAGEMENT,
    TASK_SEQUENCE_VALIDATION]) then
    (action=[DECOMPOSE_TASK,
    VALIDATE_PRECONDITIONS,
    EXECUTE_SUBTASK]) with
    strength=MANDATORY

3. If the task involves converting a
    monetary target between two
    currencies, then ensure the base
    currency matches the source of the
    monetary target and the target
    currency matches the destination
    specified in the query context. Do
    not reverse these roles, as it will
    misalign the output with the
    intended goal.
if (domain=CURRENCY_CONVERSION and
    qualifier=[CURRENCY_CONSISTENCY,
    PRECONDITION_VALIDATION]) then
    (action=[MATCH_CURRENCY,
    VALIDATE_PRECONDITIONS]) with
    strength=MANDATORY

4. If a query requires identifying
    entities from a complete set (e.g.,
    all available options), then
    retrieve the comprehensive set
    explicitly using tools designed for
    global enumeration before
    attempting subtasks that depend on
    specific entities. Do not attempt
    to infer the set by piecemeal or
    localized retrieval from individual
    components.
if (domain=ENTITY_RETRIEVAL and
    qualifier=[GLOBAL_ENUMERATION,
    DEPENDENCY_MANAGEMENT]) then
    (action=[RETRIEVE_GLOBAL_SET,
    EXECUTE_SUBTASK]) with
    strength=MANDATORY
```

Listing 7: Sample rules learned from the BFCL: Multi-Turn-Base dataset

### A.3 Use of LLMs

ChatGPT[13] is used to polish the writing of this paper.

[13] https://chatgpt.com